Reviewer Pack — Minimal Root System Length and Resolution Proof Width Inequa…

Entry a6a9b7514568 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 03:49:36 UTC

Minimal Root System Length and Resolution Proof Width Inequality

Entry ID: `a6a9b7514568` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 03:49:36 UTC

1. Conjecture

Field A (mathematical branch): Lie Theory (Root Systems) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> For every d-regular graph G, the minimal root system length (mrl(G)) of its associated Lie algebra is linearly correlated with its resolution proof width w(φ_G), such that mrl(G) = Ω(w(φ_G)).

Rationale (proposer's reasoning):

> Root systems provide a geometric representation of Lie algebras. A deeper understanding of the structure of these systems might reveal non-trivial relationships with computational complexity, particularly in the context of resolution proofs for satisfiability problems.

Taxonomy category: `Lie Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0bcb99be20e7dbfe`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture that mrl(G) = Ω(w(φ_G)), support is indicated by a correlation coefficient of r >= 0.5 for at least 25 out of 30 random seeds with p-value ≤ 0.05, and falsification occurs if r < 0.3 or any seed yields a p-value > 0.1.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'minimal root system length' AND 'resolution proof width'` - `'Lie theory' AND Boolean satisfiability` - `'root systems' IN BOOLEAN MODE AND 'resolution complexity'`

Top relevant hits considered: - [http://arxiv.org/abs/1409.0184v4] Prenilpotent pairs in the E10 root system - [http://arxiv.org/abs/1601.01857v2] Cohomology of Complements of Toric Arrangements Associated to Root Systems - [http://arxiv.org/abs/1506.05405v2] Root subsystems of rank 2 hyperbolic root systems - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/1212.5380v2] On properties of principal elements of Frobenius Lie algebras - [http://arxiv.org/abs/1908.01979v1] Non-Invasive Reverse Engineering of Finite State Machines Using Power Analysis and Boolean Satisfiability

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_d_regular_graph(d, n):
    if d * n % 2 != 0:
        return None  # Graph size must be a multiple of the degree
    graph = [[] for _ in range(n)]
    edges = set()

    def add_edge(i, j):
        if (i, j) not in edges and (j, i) not in edges:
            graph[i].append(j)
            graph[j].append(i)
            edges.add((i, j))
            edges.add((j, i))

    for i in range(n):
        for j in range(i + 1, n):
            if len(graph[i]) < d and len(graph[j]) < d:
                add_edge(i, j)

    return graph

def calculate_mrl(graph):
    # Placeholder for mrl calculation
    return sum(len(neighbors) for neighbors in graph) / len(graph)
```

```python
def generate_sat_instance(graph, variables):
    clauses = []
    for i in range(len(graph)):
        if not graph[i]:
            continue
        clause = [random.choice([1, -1]) * (i + 1)]
        for j in graph[i]:
            clause.append(random.choice([1, -1]) * (j + 1))
        clauses.append(clause)
    return clauses

def calculate_resolution_width(clauses):
    # Placeholder for resolution width calculation
    return len(max(clauses, key=len))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    d = random.randint(2, min(n - 1, 8))
    graph = generate_d_regular_graph(d, n)
    if not graph:
        return {
            "metric_name": "mrl(G)",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "Graph size must be a multiple of the degree"
        }

    mrl_G = calculate_mrl(graph)
    variables = list(range(n))
    clauses = generate_sat_instance(graph, variables)
    w_phi_G = calculate_resolution_width(clauses)

    return {
        "metric_name": "mrl(G)",
        "metric_value": mrl_G,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": mrl_G >= w_phi_G,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("metric_value" not in r or r["metric_value"] is None for r in results):
```

```
        print("RESULT: INCONCLUSIVE mapping_undefined")
    else:
        metric_values = [r["metric_value"] for r in results if "metric_value" in r and r["metric_v
        conjecture_holds = all(r["conjecture_holds"] for r in results)

        if conjecture_holds:
            mean_value = sum(metric_values) / len(metric_values)
            std_value = math.sqrt(sum((x - mean_value) ** 2 for x in metric_values) / len(metric_v
            support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)

            if support_fraction >= 0.8:
                print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={supp
            else:
                print("RESULT: INCONCLUSIVE insufficient_support")
        else:
            first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
            counterexample = "mapping_undefined"  # Placeholder
            print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={firs
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
nterexample': 'Graph size must be a multiple of the degree'}
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': 1.7142857142857142, 'instances_tested': 1, 'n_max':
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': 4.0, 'instances_tested': 1, 'n_max': 25, 'conjecture
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': 3.739130434782609, 'instances_tested': 1, 'n_max': 2
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'conjecture
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': 2.8947368421052633, 'instances_tested': 1, 'n_max':
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': 4.5, 'instances_tested': 1, 'n_max': 16, 'conjecture
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': 3.888888888888889, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': 7.578947368421052, 'instances_tested': 1, 'n_max': 1
TRIAL: {'metric_name': 'mrl(G)', 'metric_value': 5.294117647058823, 'instances_tested': 1, 'n_max': 1

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3bb53ea5.py", line 113, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3bb53ea5.py", line 113, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data that could confirm or falsify the conjecture. | next: Review and debug the test code to ensure it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17238
2	propose	ollama_remote	glm4:latest	0	0	12032
3	preregistration	ollama_remote	glm4:latest	0	0	9214
4	novelty	ollama_remote	glm4:latest	0	0	8155
5	novelty	ollama_remote	glm4:latest	0	0	10216
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15374
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18612
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14053
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10978
10	judge	ollama_remote	glm4:latest	0	0	15539

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 131410 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/a6a9b7514568.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a6a9b7514568.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a6a9b7514568.tar.gz` (if generated)